

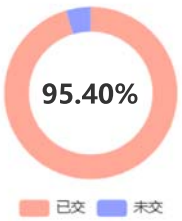

计算机系统基础

课程信息
班级管理
课程资料
教学计划
练习考试
作答分析
作业
测验
考试
统计分析
签到
证书管理
笔记
讨论区
课程评价
公告
MOOC申请

# 4.2 符号表与符号解析

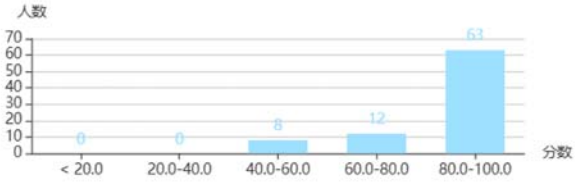
返回

题数：19 总分：100 有效时间：2024-11-11 09:52:06/2024-12-16 08:20:00 已交/总人数：83/87










作业详情 作业分析

[默认选中为正确答案]

## (一) 单选题

1，以下关于链接过程中对符号定义的判断中，错误的是（）

| | |
|---|---|
| A.全局变量声明"int x,y"中,x和y都是符号的定义 | 0.00%，0人 |
| B.全局变量声明"int *xp=&x"中，xp和x都是符号的定义 | 93.98%，78人 |
| C.静态局部变量声明"static int x=*xp"中，xp是符号的引用 | 4.82%，4人 |
| D.函数内的局部变量声明"short x=200"中，x不是符号的定义 | 1.20%，1人 |

2，若x为局部变量，xp、y和z是全局变量，则以下判断中错误的是（）

| | |
|---|---|
| A.赋值语句“int y = x+z"中，y和z都是符号的引用 | 95.18%，79人 |
| B.赋值语句“y = x+z"中，y和z都是符号的引用 | 2.41%，2人 |
| C.静态局部声明“static int x = *xp"中，xp是符号的引用 | 1.20%，1人 |
| D.赋值语句“y = x+*xp"中，y和xp都是符号的引用 | 1.20%，1人 |

3，以下有关链接符号类型的叙述中，错误的是（）

| | |
|---|---|
| A.由模块m定义并能被其他模块引用的符号称为m的全局符号 | 1.20%，1人 |
| B.由其他模块定义并被模块m引用的符号称为m的外部符号 | 2.41%，2人 |
| C.由模块m定义并仅在m中引用的符号称为m的本地符号 | 1.20%，1人 |
| D.在模块m内的某个函数中定义的局部变量称为m的局部符号 | 95.18%，79人 |

4，以下有关ELF目标文件的符号表的叙述中，错误的是（）

| | |
|---|---|
| A.可重定位和可执行两种目标文件中都有符号表且数据结构一样 | 3.61% , 3人 |
| B.符号表定义在.symtab节中，每个表项描述某个符号的相应信息 | 0.00% , 0人 |
| C.通过符号表可获得符号的名称、所在节及在节中偏移位置和长度 | 2.41% , 2人 |
| D.符号表中包含了所有定义符号的描述信息，包括局部变量的相关信息 | 93.98% , 78人 |

5，以下是有关链接过程中符号解析（符号绑定）的叙述，错误的是（）

| | |
|---|---|
| A.符号解析的目的是将符号引用与某目标模块中定义的符号建立关联 | 1.20% , 1人 |
| B.同一个符号名可能在多个模块中定义，每个定义处的符号都须分配空间 | 95.18% , 79人 |
| C.本地符号的解析比较简单，只要在本模块内定义的符号关联即可 | 1.20% , 1人 |
| D.全局符号（包括外部符号）需将模块内的引用与模块外的定义符号绑定 | 2.41% , 2人 |

6，以下有关强符号和弱符号定义的叙述中，错误的是（）

| | |
|---|---|
| A.局部符号没有强、弱符号的概念 | 1.20% , 1人 |
| B.所有出现的函数名都是强符号 | 97.59% , 81人 |
| C.没有初始化的全局变量名是弱符号 | 0.00% , 0人 |
| D.已初始化的全局变量名是强符号 | 1.20% , 1人 |

7，以下有关强符号和弱符号的符号解析的叙述中，错误的是（）

| | |
|---|---|
| A.一个符号名只有一个强符号，否则符号解析失败 | 1.20% , 1人 |
| B.一个符号名可以有多个弱符号，任选一个为其定义 | 0.00% , 0人 |
| C.一个符号名可以有一个强符号和多个弱符号，强符号为其定义 | 0.00% , 0人 |
| D.一个符号名可以仅出现在引用处或仅出现在定义处 | 98.80% , 82人 |

8，以下是两个源程序文件：

```
/* m1.c */
int p(void);
int main()
{
   int p1 = p();
   return p1;
}
```

```
/* m2.c */
static main = 1;
 int p()
{
    main++;
     return main;
  }
```

对于上述两个源程序文件链接时的符号解析，错误的是（）

| | |
|---|---|
| A.在m1中，定义了一个强符号main和一个弱符号p | 2.41% , 2人 |

B.在m2中，定义了一个强符号p和一个本地符号main — 0.00% , 0人

C.在m1中，对m2中定义的强符号p的引用只有一处 — 0.00% , 0人

D.因为出现了两个强符号main，所以会发生链接错误 — 97.59% , 81人

9，以下是两个源程序文件：

```
/* m1.c */                  /* m2.c */
int p(void);                int main = 1;
int main()                  int p()
{                           {
   int p1 = p();               int p1 = main++;
   return p1;                  return main;
}                           }
```

对于上述两个源程序文件链接时的符号解析，错误的是（）

A.在m1中，定义了一个强符号main和一个弱符号p — 2.41% , 2人

B.在m2中，定义了一个强符号p和一个强符号main — 4.82% , 4人

C.在模块m1的所有语句中，对符号p1的引用共有两处 — 81.93% , 68人

D.因为出现了两个强符号main，所以会发生链接错误 — 10.84% , 9人

10，以下是两个源程序文件：

```
/* m1.c */                  /* m2.c */
int x = 100;                float x;
int p1(void);               static main = 1;
int main()                  int p1()
{                           {
    x = p1();                 int p1 = main + (int) x;
   return x;                  return p1;
}                           }
```

对于上述两个源程序文件链接时的符号解析，错误的是（）

A.m1中对x的两处引用都与m1中对x的定义绑定 — 8.43% , 7人

B.m2中对x的引用与m2中对x的定义绑定 — 84.34% , 70人

C.m2中的变量p1与函数p1被分配在不同存储区 — 3.61% , 3人

D.虽然x、main和p1都出现了多次定义，但不会发生链接错误 — 3.61% , 3人

11，以下是两个源程序文件：

```
/* m1.c */                  /* m2.c */
#include
int x = 100;                double x;
short y = 1,z = 2;
int main()                  int p1()
{                           {
    p1();                       x = -1.0;
    printf("x=%d,z=%d\n",x,z);
}                           }
```

上述程序执行的结果是（）（提示：1074790400=2^30+2^20，16400=2^14+2^4)

A.x = 100,z = 2 — 1.20% , 1人

B.x = -1,z = 2 — 3.61% , 3人

C.x = -1074790400,z = 0 — 3.61% , 3人

D.x = 0,z = -16400 — 90.36% , 75人

12，假设调用关系如下：func.o → libx.a 和liby.a中的函数，libx.a→ libz.a中的函数，libx.a和liby.a之间、liby.a和libz.a相互独立，则以下几个命令行中，静态链接发生错误的命令是（）

| | |
|---|---|
| A.gcc -static -o myfunc func.o libx.a liby.a libz.a | 0.00% , 0人 |
| B.gcc -static -o myfunc func.o liby.a libz.a libx.a | 90.36% , 75人 |
| C.gcc -static -o myfunc func.o libx.a libz.a liby.a | 3.61% , 3人 |
| D.gcc -static -o myfunc func.o liby.a libx.a libz.a | 6.02% , 5人 |

13，假设调用关系如下：func1.o → func2.o，func1.o → libx.a中的函数，func2.o → libx.a中的函数，libx.a → liby.a同时liby.a → libx.a，则以下几个命令行中，能够正确进行静态链接的命令是（）

| | |
|---|---|
| A.gcc -static -o myfunc func1.o func2.o libx.a liby.a libx.a | 87.95% , 73人 |
| B.gcc -static -o myfunc func2.o func1.o liby.a libx.a liby.a | 1.20% , 1人 |
| C.gcc -static -o myfunc libx.a liby.a libx.a func1.o func2.o | 1.20% , 1人 |
| D.gcc -static -o myfunc liby.a libx.a liby.a func1.o func2.o | 9.64% , 8人 |

## (三) 判断题

1，假定a和b是可重定位目标文件或静态库文件，a->b表示b中定义了一个被a引用的符号。现有引用关系“p.o->libx.a->liby.a->p.o”，则使得链接器能够解析所有符号引用的最短命令行（含有最少数量的可重定位目标文件或静态文件参数），为gcc -static -o p p.o libx.a liby.a p.o

| | |
|---|---|
| yes | 93.98% , 78人 |
| no | 6.02% , 5人 |

2，假定a和b是可重定位目标文件或静态库文件，a->b表示b中定义了一个被a引用的符号。现有引用关系“p.o->libx.a->liby.a，同时liby.a->libx.a”，则使得链接器能够解析所有符号引用的最短命令行（含有最少数量的可重定位目标文件或静态文件参数），为gcc -static -o p p.o libx.a liby.a libx.a

| | |
|---|---|
| yes | 91.57% , 76人 |
| no | 7.23% , 6人 |

3，假定a和b是可重定位目标文件或静态库文件，a->b表示b中定义了一个被a引用的符号。现有引用关系“p.o->libx.a->liby.a->libz.a，同时liby.a->libx.a->libz.a”，则使得链接器能够解析所有符号引用的最短命令行（含有最少数量的可重定位目标文件或静态文件参数），为gcc -static -o p p.o libx.a liby.a libz.a

| | |
|---|---|
| yes | 6.02% , 5人 |
| no | 93.98% , 78人 |


Copyright © 2016-2025 浙江省高等学校在线开放课程共享平台　　浙ICP备05018780号-10　　浙公网安备 33010602010741号
关注在浙学微信公众号